DATA ANALYTICS | NATURAL LANGUAGE PROCESSING | BUSINESS INTELLIGENCE

568,454 REVIEWS.

What can 568,454 customer reviews tell us that a star rating can't?

Customer Review Analytics & Sentiment Intelligence

A Python + NLP workflow that transforms unstructured Amazon review text into sentiment signals, complaint themes, error-analysis outputs, and product-level decision support.

PROJECT STACK

Python | Pandas | NLTK VADER | Tokenization | Matplotlib | CSV / DataFrames

END-TO-END WORKFLOW

568K RAW — source data → **300 BALANCED** — stratified sample → **NLP SCORING** — VADER → **ERROR ANALYSIS** — agreement + failures → **BI INSIGHTS** — themes + actions

Team project • MIT 5032 Analytics Programming • University of Oklahoma

568K+ raw reviews → a reproducible test set

We used stratified random sampling so positive-heavy source data would not hide performance on neutral and negative reviews.

568,454
SOURCE REVIEWS
74,258 products • 1999–2012

10
PRODUCTS
Selected for balanced coverage

300
FINAL SAMPLE
30 reviews per product

CLASS-BALANCED DISTRIBUTION



SAMPLING LOGIC

ProductId × RatingSentiment
→ random sample within each stratum
→ concatenate into final evaluation set

random_state=42 → deterministic / reproducible sample

REAL PROJECT CODE • STRATIFIED SAMPLING

```
for product in PRODUCTS:
    product_reviews = df[df["ProductId"]
== product]

    for sentiment in ["negative",
"neutral", "positive"]:
        group = product_reviews[

product_reviews["RatingSentiment"] ==
sentiment
        ]
        sample = group.sample(n=10,
random_state=42)
```

WHY THIS MATTERS

Balanced classes make the evaluation interpretable: 100 positive + 100 neutral + 100 negative. The sample is designed for testing the workflow — not to represent Amazon's natural sentiment distribution.

Under the hood: text → sentiment + complaint signals

The application combines deterministic sentiment scoring, text preprocessing, rule-based theme classification, and structured outputs in one explainable workflow.

PROCESSING ARCHITECTURE



NLTK VADER • SENTIMENT CLASSIFICATION

```
scores = sia.polarity_scores(text)
compound = scores["compound"]

if compound >= 0.05:
    return "positive"
elif compound <= -0.05:
    return "negative"
else:
    return "neutral"
```

TOKENIZATION + STOP-WORD FILTERING

```
tokens = wordpunct_tokenize(text.lower())
clean_list = []

for word in tokens:
    if word.isalpha() and word not in
stop_words:
        clean_list.append(word)
```

LEXICAL THEME FEATURES

```
"taste" → taste, flavor, sweet, salty,
bitter
"packaging" → package, box, seal, damaged
"quality" → quality, texture, dry, hard,
soft
```

STRUCTURED OUTPUTS

```
VADER_Compound
VADER_Sentiment
Theme
Match
TopComplaint + SuggestedAction
```

Technical pattern: **unstructured text → deterministic NLP features → structured analytics output**

The interesting part was where the classifier disagreed

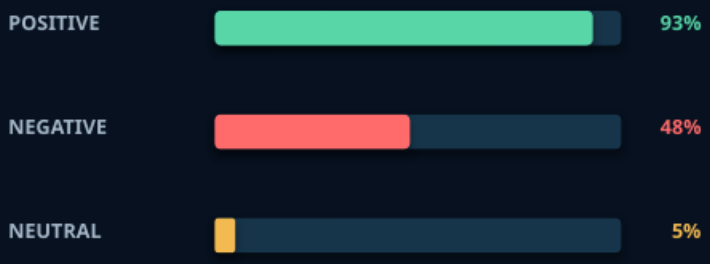
We compared VADER's text-based sentiment with labels derived from star ratings — then inspected disagreement cases instead of hiding them.

48.67%
OVERALL AGREEMENT
146 of 300 reviews matched

154
DISAGREEMENTS
Saved to failure_cases.csv

3
CLASSES
100 positive • 100 neutral • 100 negative

CLASS-CONDITIONAL AGREEMENT



EVALUATION + ERROR ANALYSIS

```
df["Match"] = (
    df["RatingSentiment"] ==
df["VADER_Sentiment"]
)

accuracy = df["Match"].mean() * 100
failures = df[df["Match"] == False].copy()

df.groupby("RatingSentiment")["Match"]
  .mean().mul(100).round(2)
```

FAILURE-MODE INSIGHT

Mixed reviews can contain strong positive words inside an overall negative experience. Neutral 3-star reviews are especially difficult because text tone may not be neutral.

OUTPUT ARTIFACT

```
failure_cases.csv
→ inspect disagreements
→ identify edge cases
```

Key lesson: **evaluation is more useful when failure cases become part of the analysis.**

From NLP outputs to product-level decision support

The goal was not just a sentiment score — it was to surface recurring customer issues that a seller could investigate.

NEGATIVE-REVIEW COMPLAINT THEMES



PRODUCT-LEVEL OUTPUT SCHEMA

```
ProductId | TopComplaint | Count |
SuggestedAction

B0090X8IPM | taste      | 9 | Review
taste/flavor
B001EO5Q64 | packaging  | 5 | Check
packaging/sealing
B007JFMH8M | quality    | 6 | Check
quality/consistency
```

BUSINESS TAKEAWAY

Taste was the most common complaint theme in the sample. Five of the 10 products were taste-led, while others surfaced packaging, quality, price/value, or uncategorized issues.

Unstructured feedback → structured signals → product-level action.

GROUPBY-STYLE AGGREGATION • TOP ISSUE / PRODUCT

```
counts =
product_data["Theme"].value_counts()
top_theme = counts.index[0]
count = counts.iloc[0]

product_list.append({
    "ProductId": product,
    "TopComplaint": top_theme,
    "Count": count
})
```

SKILLS DEMONSTRATED

Python • Pandas • DataFrames • Stratified Sampling • NLP • VADER • Tokenization • Text Preprocessing • Rule-Based Classification • EDA • Error Analysis • Matplotlib • Business Intelligence